



Programovacie jazyky

Funktory a aplikatívne funktory

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

19. apríla 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Funktory

Motivácia

- máme implementované „zložité“ funkcie `inc` a `len`
 - po „značnej“ námahe sme dospeli k implementáciám:
 - `inc x = x + 1`, alebo `inc = (+1)`
 - `{len [] = 0; len (_:xs) = inc $ len xs}`
- aby sa „investícia“ do implementácií vrátila, snažíme sa ich použiť vždy, keď potrebujeme inkrementovať nejaké číslo alebo zistiť dĺžku zoznamu
- v novom projekte vznikla požiadavka využiť tieto funkcie v kontexte **Maybe**, teda aby sme `inc` a `len` vedeli aplikovať aj na **Maybe Int** a **Maybe [a]**
 - `inc $ Just 10` *-- nie je možné*
 - `len $ Just [2..10]` *-- nie je možné*

Funkcie `incm` a `lenm` pre `Maybe`

```
1 incm :: Maybe Int -> Maybe Int
2 incm Nothing = Nothing
3 incm (Just x) = Just (inc x)
```

```
1 lenm :: Maybe [a] -> Maybe Int
2 lenm Nothing = Nothing
3 lenm (Just x) = Just (len x)
```

- príliš sa opakujeme
 - 2. riadok: úplne rovnaký
 - 3. riadok: jediná zmena je výmena `inc` $\leftrightarrow$ `len`
- preto tento proces konverzie funkcie „`f :: a -> b`“ na funkciu „`fm :: Maybe a -> Maybe b`“ extrahujeme do univerzálnej funkcie `tom`

Funkcie incm a lenm pomocou funkcie tom

```
1 tom :: (a -> b) -> Maybe a -> Maybe b
2 tom _ Nothing = Nothing
3 tom f (Just xs) = Just (f xs)
```

- funkcia `tom` nám umožňuje „pozdvihnúť“ (lift) funkciu `f :: a -> b` na funkciu `fm :: Maybe a -> Maybe b`

```
1 incm = tom inc
2 lenm = tom len
```

- v ďalšom projekte vznikla potreba aplikovať funkcie `inc` a `len` na jednotlivé prvky zoznamov
 - `inc [1..3]` *-- nie je možné*
 - `len [[1..3],[2..7],[1,7]]` *-- nie je možné*

Funkcie incl a lenl pre zoznam

```
1 incl :: [Int] -> [Int]
2 incl [] = []
3 incl (x:xs) = (inc x) : incl xs
```

```
1 lenl :: [[a]] -> [Int]
2 lenl [] = []
3 lenl (x:xs) = (len x) : lenl xs
```

- opäť sa príliš opakujeme
 - 2. riadok: úplne rovnaký
 - 3. riadok: jediná zmena je výmena `inc` $\leftrightarrow$ `len`
- preto tento proces konverzie funkcie „`f :: a -> b`“ na funkciu „`fl :: [a] -> [b]`“ extrahujeme do univerzálnej funkcie `tol`

Funkcie `incl` a `lenl` pomocou funkcie `tol`

```
1 tol :: (a -> b) -> [a] -> [b]
2 tol _ [] = []
3 tol f (x:xs) = (f x) : tol f xs
```

- funkcia `tol` nám umožňuje jednoducho „pozdvihnúť“ funkciu `f :: a -> b` na funkciu `fl :: [a] -> [b]`

```
1 incl = tol inc
2 lenl = tol len
```

- majú funkcie `tom` a `tol` niečo spoločné?

Funkcie tom a tol

```
1 tom :: (a -> b) -> Maybe a -> Maybe b
2 tom _ Nothing = Nothing
3 tom g (Just xs) = Just (g xs)
```

```
1 tol :: (a -> b) -> [a] -> [b]
2 tol _ [] = []
3 tol g (x:xs) = (g x) : tol g xs
```

- funkcie `tom` a `tol` majú spoločnú štruktúru signatúry:
 - $(a \rightarrow b) \rightarrow f \ a \rightarrow f \ b$, pričom pre:
 - `tom` je $f = \text{Maybe}$
 - `tol` je $f = []$ („`[a]`“ je to isté ako „`[]` a“)
- implementácia sa prípad od prípadu líši, podľa f
 - využijeme ad-hoc polymorfizmus


```
1 class Functor (f :: * -> *) where
2   fmap :: (a -> b) -> f a -> f b
3   (<$) :: a -> f b -> f a
4   (<$) = fmap . const
5   {-# MINIMAL fmap #-}
```

Inšancie sú: (**Either** a), [], **Maybe**, **IO**, ((->) a) a ((,) a).

Ak je typový konštruktor `f :: * -> *` inštanciou typovej triedy **Functor**, tak pomocou `fmap` môžeme funkciu `g :: a -> b`:

- aplikovať na hodnoty typu `a` v inštancii „`x :: f a`“
`fmap g x :: f b`
- „pozdvihnúť“ (lift) na funkciu `fmap g :: f a -> f b`

Implementácia `fmap` má spĺňať nasledovné identity:

- `fmap id = id`
- `fmap (f . g) = fmap f . fmap g`

Maybe, (Either e), ((,) a), [] a ((->) a) ako funktor

```
1 instance Functor Maybe where
2   fmap _ Nothing = Nothing
3   fmap f (Just a) = Just (f a)
```

```
1 instance Functor (Either e) where
2   fmap _ (Left x) = Left x
3   fmap f (Right y) = Right (f y)
```

```
1 instance Functor ((,) a) where
2   fmap f (x, y) = (x, f y)
```

```
1 instance Functor [] where
2   fmap = map
```

```
1 instance Functor ((->) a) where
2   fmap = (.)
```

Ďalšie funkcie pre prácu s funktormi

```
import Data.Function
```

```
1 ($>) :: Functor f => f b -> a -> f a
2 ($>) = flip (<$)
3
4 (<$>) :: Functor f => (a->b)->f a->f b
5 (<$>) = fmap
6
7 void :: Functor f => f a -> f ()
8 void x = () <$ x
```

Príklady

```
1 fmap (+1) $ Left 10 -- Left 10
2 fmap (+1) $ Right 10 -- Right 11
3 (+1) <$> Left 10 -- Left 10
4 (+1) <$> Right 10 -- Right 11
5 'a' <$ [1..4] -- "aaaa"
6 void (10, True) -- (10, ())
```

Výsledok snaženia

Začínali sme s tým, že nemôžeme aplikovať `inc` a `len` na argument zabalený v nejakom kontexte:

- `inc $ Just 10` *-- nie je možné*
- `len $ Just [2..10]` *-- nie je možné*
- `inc $ [1..3]` *-- nie je možné*
- `len $ [[1..3],[2..7],[1,7]]` *-- nie je možné*

to síce stále nie je možné, ale stačí nahradiť `$` za `<$>` a dostávame:

- `inc <$> Just 10` *-- Just 11*
- `len <$> Just [2..10]` *-- Just 9*
- `inc <$> [1..3]` *-- [2,3,4]*
- `len <$> [[1..3],[2..7],[1,7]]` *-- [3,6,2]*

Vďaka `<$>` (`fmap`) môžeme aplikovať ľubovoľnú funkciu vo vnútri ľubovoľného funktora (nie len `Maybe`, `[]`). Funkcia `fmap` umožňuje modifikovať obsah prvkov kontajnera bez zmeny jeho tvaru.

Automatické odvodenie inštancie triedy Functor pre List

```
1 {-# LANGUAGE DeriveFunctor #-}  
2 data List a = Empty  
3             | Cons a (List a)  
4             deriving (Show, Functor)
```

```
1 list :: List Int  
2 list = Cons 10 (Cons 7 (Cons 2 Empty))
```

- $(+1) \langle \$ \rangle \text{ list} \rightsquigarrow \text{Cons } 11 (\text{Cons } 8 (\text{Cons } 3 \text{ Empty}))$

Automatické odvodenie inštancie triedy Functor pre Error

```
1 {-# LANGUAGE DeriveFunctor #-}  
2 data Error e v = LTT v  
3                 | Skoro e v e  
4                 | Error e  
5                 | OK v v  
6                 deriving (Show, Functor)  
7  
8 el :: [Error String Integer]  
9 el = [ Error "E1"  
10       , OK 20 30  
11       , LTT 0  
12       , Skoro "E" 10 "7"]
```

- „funkcia sa aplikuje cez posledný typový parameter“
- `fmap (fmap (+1)) el`
 $\rightsquigarrow$ `[Error "E1", OK 21 31, LTT 1, Skoro "E" 11 "7"]`
 - zvykne sa písať aj ako: `(fmap . fmap) (+1) el`

Aplikatívne funktory

Motivácia

- problém s aplikovaním (+1) na hodnotu uloženú v **Maybe** sme vyriešili pomocou **fmap** (alebo infixovým ekvivalentom <\$>):
 - (+1) <\$> **Just** 10 $\rightsquigarrow$ **Just** 11
- teraz nastala situácia, keď potrebujeme aplikovať binárnu funkciu na dve hodnoty uložené v kontexte **Maybe**, napríklad:
 - (++) <\$> **Just** "AB" <\$> **Just** "BA" -- *nedá sa*
 - **x** = (++) <\$> **Just** "AB"
x :: **Maybe** (**String** -> **String**)
 - teraz máme čiastočne aplikovanú funkciu uloženú vo vnútri **Maybe** a potrebovali by sme ju aplikovať na ďalšiu hodnotu v kontexte **Maybe**, napríklad **Just** "BA"
- ako na to?
 - **x** <\$> **Just** "BA" nepomôže, lebo (<\$>)
 - je typu: (a -> b) -> f a -> f b
 - ale potrebujeme: **f** (a -> b) -> f a -> f b

Riešenie konkrétneho problému

```
1 infixl 4 <*>
2 (<*>) :: Maybe (a->b)->Maybe a->Maybe b
3 Just f <*> Just x = Just (f x)
4 _ <*> Nothing = Nothing
5 Nothing <*> _ = Nothing
```

- teraz môžeme dokončiť predošlý príklad:

```
x = (++) <$> Just "AB"
```

```
x <*> Just "BA" ~== Just "ABBA"
```

- inak povedané namiesto `((++) $ "AB") $ "BA"` môžeme v kontexte **Maybe** písať veľmi podobný zápis:
 - `((++) <$> Just "AB" <*> Just "BA"`
- pre rôzne kontexty je síce štruktúra signatúry funkcie `(<*>)` rovnaká, ale jej implementácia je rôzna
 - opäť využijeme ad-hoc polymorfizmus

Aplikatívny funktor

```
1 class Functor f => Applicative f where
2   pure :: a -> f a
3   (<*>) :: f (a -> b) -> f a -> f b
4   (<*) :: f a -> f b -> f a
5   (*>) :: f a -> f b -> f b
6   {-# MINIMAL pure, (<*>) #-}
7   a1<*a2=pure const <*> a1 <*> a2 -- a1
8   a1*>a2=pure (const id)<*>a1<*>a2-- a2
```

(**Either** e), **[]**, **Maybe**, **IO**, ((**->**) a) a **Monoid** a=>((**,**) a)

- **pure** – vloží „čistú“ hodnotu do kontextu
- (<*>) – aplikuje funkciu v kontexte na hodnotu v kontexte

Implementácia **pure** a (<*>) má spĺňať nasledovné identity:

- **pure** id <*> v = v (← identity) (↓ composition)
- **pure** (.) <*> u <*> v <*> w = u <*> (v <*> w)
- **pure** f <*> pure x = **pure** (f x) (homomorphism)
- u <*> pure y = **pure** (\$ y) <*> u (interchange)

Maybe, (Either e) a [] ako aplikatívny funktor

```
1 instance Applicative Maybe where
2   pure          = Just
3   Just f  <*> m = fmap f m
4   Nothing <*> _ = Nothing
```

```
1 instance Applicative (Either e) where
2   pure          = Right
3   Left e  <*> _ = Left e
4   Right f <*> x = fmap f x
```

```
1 instance Applicative [] where
2   pure x      = [x]
3   fs <*> xs = [f x | f <- fs, x <- xs]
```

- $[(+), (*)] \text{ <*> } [1, 2] \text{ <*> } [10, 20]$
 $\rightsquigarrow [11, 21, 12, 22, 10, 20, 20, 40]$

`((,) a) a` (`(->) a`) ako aplikatívny funktor

```
1 instance Monoid a =>
2     Applicative ((,) a) where
3     pure x = (mempty, x)
4     (u, f) <*> (v, x) =
5         (u 'mappend' v, f x)
```

- `("a", (+)) <*> ("b", 10) <*> ("c", 20) ~> ("abc", 30)`

```
1 instance Applicative ((->) a) where
2     pure      = const
3     (f <*> g) x = f x (g x)
```

- `z = ((+) <$> (+10) <*> (*2)) :: Int -> Int`
 - `z 10 ~> 40` `z 11 ~> 43` `z 12 ~> 46`
- `f = \k -> if k `mod` 3 == 0 then (+) else (*)`
`z = (f <*> (+10) <*> (*2)) :: Int -> Int`
 - `z 10 ~> 400` `z 11 ~> 462` `z 12 ~> 46`
- $S = (\langle * \rangle)$, $K = \text{pure}$; $(\langle * \rangle) \text{ const undefined} \equiv \text{id} \Rightarrow I = SKK$

`((,) a) a` (`(->) a`) ako aplikatívny funktor

```
1 instance Monoid a =>
2       Applicative ((,) a) where
3   pure x = (mempty, x)
4   (u, f) <*> (v, x) =
5       (u 'mappend' v, f x)
```

- `("a", (+)) <*> ("b", 10) <*> ("c", 20) ~> ("abc", 30)`

```
1 instance Applicative ((->) a) where
2   pure      = const
3   (f <*> g) x = f x (g x)
```

- `z = ((+) <*> (+10) <*> (*2)) :: Int -> Int`
 - `z 10 ~> 40` `z 11 ~> 43` `z 12 ~> 46`
- `f = \k -> if k `mod` 3 == 0 then (+) else (*)`
`z = (f <*> (+10) <*> (*2)) :: Int -> Int`
 - `z 10 ~> 400` `z 11 ~> 462` `z 12 ~> 46`
- $S = (\langle * \rangle)$, $K = \text{pure}$; $(\langle * \rangle) \text{ const undefined} \equiv \text{id} \Rightarrow I = SKK$